

GenAI, RAG, Agents & Multi-Agent Systems

A complete primer on how these four layers build on each other, from base model to coordinated agent teams.

- Generative AI - the base model that produces text, code, images
- RAG - grounds that model in external, up-to-date knowledge
- Agents - let the model take actions and use tools, not just answer
- Multi-agent systems - coordinate several agents toward one goal

The Four-Layer Stack

HOW THEY RELATE

- Each layer sits on top of the previous one - you don't replace GenAI, you extend it
- GenAI generates. RAG grounds. Agents act. Multi-agent coordinates.
- Most production systems today use two or more layers together

WHEN TO STOP AT EACH LAYER

- GenAI alone: static tasks, no external knowledge needed (writing, brainstorming)
- GenAI + RAG: knowledge changes often, citations matter
- GenAI + RAG + Agents: task requires taking actions, not just answering
- Multi-agent: task is complex enough to benefit from role specialization

Generative AI: What It Is

CORE IDEA

- A model trained to predict the next token, pixel, or frame given context
- Trained on massive datasets to learn patterns in language, code, images
- Produces new content by sampling from what it learned, not by lookup

MODEL FAMILIES

- Large language models (LLMs) - text and code generation
- Diffusion models - image and video generation
- Multimodal models - handle text, image, and audio in one model

Generative AI: The Core Limitation

FROZEN KNOWLEDGE

- Everything the model knows was fixed at training time
- No awareness of events, documents, or data created after that cutoff
- No access to private, internal, or company-specific information at all

HALLUCINATION

- The model can produce fluent, confident, and wrong answers
- It has no built-in way to say 'I don't actually know this'
- This is the single biggest reason RAG exists as a pattern

Generative AI: Where It's Strong Alone

GOOD FIT WITHOUT RAG OR AGENTS

- Summarizing or rewriting text the user already provides
- Brainstorming, ideation, drafting from a prompt
- Code generation for common, well-known patterns
- Translation and tone/style transformation

POOR FIT WITHOUT HELP

- Answering questions about private or recent data
- Anything requiring the model to check a fact before answering
- Multi-step tasks requiring real-world actions

RAG: Giving GenAI Real Knowledge

THE FOUR-STEP FLOW

- Query - the user asks a question
- Retrieve - relevant documents are found from an external source
- Augment - retrieved content is added into the prompt as context
- Generate - the model answers grounded in that retrieved context

WHY IT WORKS

- Answers become traceable to a specific source, not model memory
- Knowledge can be updated by changing the data source, not the model
- Far cheaper than retraining or fine-tuning for new knowledge

RAG: The Core Components

PIPELINE

- Data sources - documents, databases, APIs, PDFs, web pages
- Document processing - text extraction, cleaning, chunking (500-1000 tokens)
- Embedding generation - converts chunks into vectors
- Vector storage - Pinecone, Weaviate, pgvector, Qdrant
- Retrieval - similarity search, filtering, ranking
- LLM integration - prompt construction and generation
- Monitoring - quality and performance tracking

RAG: Types of Vector Search

DENSE (SEMANTIC)

- Embeds query and docs into vectors, compares by similarity
- Strong at paraphrase and meaning matches, weak on exact terms

SPARSE (LEXICAL)

- BM25/TF-IDF - matches exact terms, codes, product names
- No semantic understanding, but no embedding cost

HYBRID

- Combines dense and sparse, merges scores for better recall

MULTI-VECTOR

- ColBERT-style - one vector per token instead of per chunk
- Higher accuracy, higher cost - reserve for high-value queries

RAG: Optimization Strategies

RETRIEVAL QUALITY

- Hybrid search - combine semantic and keyword search (~30% accuracy gain)
- Query rewriting - expand and reformulate the user's query
- Reranking - cross-encoder models rerank the top-K results
- Contextual compression - strip irrelevant content from retrieved chunks

SPEED AND COST

- Caching frequent queries and embeddings (up to 10x faster)
- Feedback loops - improve ranking from real usage over time
- A/B testing - measure real production performance of changes

RAG: How to Measure Success

RETRIEVAL METRICS

- Precision@K and Recall@K - relevance and coverage in top results
- MRR (mean reciprocal rank)
- Target: above 80% precision, above 70% recall

GENERATION METRICS

- Faithfulness - does the answer match the retrieved sources
- Answer relevancy and context precision
- Target: above 85% faithfulness

TOOLING

- RAGAS, TruLens, Phoenix for automated evaluation and monitoring

Agents: From Answering to Acting

WHAT CHANGES

- The model can call tools - search, code execution, APIs, file access
- It plans a sequence of steps instead of returning one response
- It can observe the result of an action and decide the next step
- It can check its own output and retry or self-correct

WHY THIS MATTERS

- Many real tasks require doing something, not just describing it
- Agents close the gap between 'the model knows the answer' and 'the task is done'

Agents: Core Components

THE AGENT LOOP

- Planning - break a goal into a sequence of steps
- Tool use - call external functions, APIs, or code execution
- Memory - short-term (current task) and long-term (past sessions)
- Reflection - evaluate its own output and revise if needed

COMMON PATTERN: ReAct

- Reason about what to do next, then Act by calling a tool
- Observe the result, then reason again - loop until the goal is met

Agents: Common Agent Types

REACTIVE AGENTS

- Respond directly to input with no internal planning step
- Fast and simple, but brittle on multi-step tasks

PLANNING AGENTS

- Build an explicit multi-step plan before acting
- Better for tasks with dependencies between steps

TOOL-USING / REACT AGENTS

- Interleave reasoning and tool calls in a loop
- Most common pattern in production coding and research agents

Agentic RAG: Retrieval as a Tool

THE 2026 SHIFT

- Retrieval is no longer a fixed step before every generation
- The agent decides IF and WHEN retrieval is needed per query
- Agents can re-query when the first retrieval pass leaves gaps (multi-hop)
- Tool-calling loops can replace a fixed retrieve-then-generate pipeline

WHY IT MATTERS

- Reduces unnecessary retrieval calls on simple queries
- Handles multi-part questions single-pass RAG misses
- Frameworks: LangGraph, LlamaIndex Agents, custom agent loops

Multi-Agent Systems: Specialization at Scale

WHY MULTIPLE AGENTS

- One agent per role - researcher, coder, reviewer, planner
- A coordinator/orchestrator agent routes work and merges results
- Mirrors how human teams split large tasks into specialties

WHEN IT'S WORTH IT

- Task is complex enough to have genuinely distinct sub-tasks
- Different steps benefit from different tools, prompts, or models

Multi-Agent: Common Architectures

ORCHESTRATOR-WORKER

- One coordinator agent assigns tasks to specialized worker agents
- Coordinator merges results into a final output

HIERARCHICAL

- Multiple layers of agents - managers oversee sub-teams of agents
- Useful for very large, deeply decomposable tasks

PEER-TO-PEER

- Agents communicate directly with each other, no central coordinator
- More flexible, but harder to debug and control

Multi-Agent: Communication Patterns

HOW AGENTS SHARE STATE

- Shared memory/blackboard - all agents read and write to one store
- Message passing - agents send structured messages to each other
- Shared task queue - agents pick up and complete discrete tasks

PROTOCOLS AND STANDARDS

- Structured message formats (JSON/schema-based) reduce miscommunication
- Explicit handoff points prevent two agents from working the same task

Multi-Agent: Tradeoffs and Challenges

COSTS

- Higher coordination overhead and latency than a single agent
- More tokens spent on inter-agent communication and planning

FAILURE MODES

- Failure in one agent can cascade if there's no review step
- Agents can loop or disagree with no natural termination point
- Debugging is harder - failures can originate in coordination, not logic

Choosing the Right Layer

GENAI ALONE

- Best for: static, self-contained tasks with no external knowledge need
- Cost: lowest - single model call

GENAI + RAG

- Best for: knowledge that changes often, source citations required
- Cost: moderate - retrieval plus generation

GENAI + RAG + AGENTS

- Best for: tasks that require taking action, not just answering
- Cost: higher - multiple reasoning and tool-call steps

MULTI-AGENT

- Best for: complex, multi-part tasks with distinct specialties
- Cost: highest - coordination and multiple agent loops

Putting It Together: A Practical Path

START SIMPLE

- Ship with GenAI alone if the task truly needs no external knowledge
- Add RAG the moment answers need to be current or cite a source

ADD CAPABILITY AS NEEDED

- Add agent tool-use only when the task requires real actions
- Move to multi-agent only once a single agent's task list is genuinely too broad for one role

THE COMMON MISTAKE

- Reaching for multi-agent complexity before a simpler layer has been tried
- Start at the lowest layer that solves the actual problem, then add layers only when a concrete limitation shows up